Throughput-Probed Bandit (TPB): Zero-Hot-Path-Overhead Adaptive Promotion for Predictive-Translation Buffer Pools

Anonymous Author(s)
Reproduction-with-extension report on PrediCache (SIGMOD 2026)

Abstract

PrediCache (SIGMOD 2026) introduces predictive translation: each page is bound to a deterministic preferred buffer frame, letting the CPU speculatively read from that frame in parallel with the hash-table lookup. Accuracy is maintained by probabilistically promoting hot pages into their preferred frames with hand-picked constants ($\frac{1}{32}$ for the no-displace case, $\frac{1}{512}$ otherwise). We show empirically that these constants are workload-dependent and introduce TPB, an online controller that adapts the probability using the system's own per-second transaction counter as its only signal. TPB's key architectural property is zero hot-path overhead: a single global atomic-relaxed load per buffer access, identical to the static default. All controller logic lives on the existing stat thread. Across TPC-C, uniform random-read, and skewed YCSB ($\zeta = 0.9$), TPB matches the hand-picked default within statistical noise without manual per-workload tuning, eliminating a 22 % regression that an earlier non-architectural variant suffered on skewed reads. We also document the v3→v4→v5 development arc: the lessons learned about where to measure are more general than TPB itself.

1 Introduction

Buffer-pool design has been driven for a decade by a tight efficiency target: each page-id (PID)-to-frame translation should cost no more than the in-memory case [3, 4]. The hash-table-based pools of MySQL, PostgreSQL, and SQLite carry two dependent memory accesses per page request (chained look-up plus header dereference), which has historically been seen as a fundamental bottleneck [3, 4]. This motivated pointer-swizzling (LeanStore [3]) and OS-page-table designs (vmcache [4]), each of which sacrifices a qualitative property in exchange for speed.

PrediCache [1] closes this gap without either trade-off. Every PID is bound at startup to a deterministic preferred frame (hash(pid) mod frameCount). On a page request, the CPU speculatively reads from the predicted frame in parallel with the hash-table look-up; superscalar execution hides one of the two memory accesses. When the prediction is correct, two dependent accesses are completed in the time of one. Accuracy of the speculation is maintained by probabilistic promotion: on the second and later accesses to a page, it is moved to its preferred frame with fixed probability $p_0 = \frac{1}{32}$, or with probability $p_1 = \frac{1}{512}$ when the move requires displacing an incumbent page [1].

These two constants are the only knobs in PrediCache that are not justified by a first-principles argument in the paper. Section 3 shows empirically that $p_0 = \frac{1}{32}$ is suboptimal by 1–2 % on TPC-C and uniform random-read, and exactly optimal on skewed YCSB at $\zeta = 0.9$. In other words, the right p_0 is workload-dependent.

A naïve adaptation strategy — per-access counters, hill-climbing on hit-rate — fails catastrophically in this regime. Our earlier attempts (CBP-v3, CBP-v4, TPB-v1 …v3) ranged from −22.4 % to −2.2 % versus the static default on a 30-second TPC-C run; the worst case was significant at $|t| = 4.48$. Two failure modes dominated: (i) per-access bookkeeping (atomic counter increments, sampling) imposes 8–23 % throughput tax even when no adaptation runs; (ii) hit-rate is monotonic in $\log_2(1/p_0)$, so any bandit that uses hit-rate as its reward overshoots into one of the extremes (always-promote or never-promote) rather than stopping at the throughput optimum.

Contribution of this paper. We design and evaluate TPB (Throughput-Probed Bandit), an adaptive promoter that solves both failure modes by a single architectural choice: the instrumentation lives outside the buffer-pool access path entirely. TPB uses a three-window probe-sandwich $[\mathrm{lc}_{\mathrm{base}}, \mathrm{lc}_{\mathrm{probe}}, \mathrm{lc}_{\mathrm{base}}]$ fed by PrediCache's own per-second stat thread; the entire controller state machine is ≈ 50 lines of C++ and runs once per second on a single thread. Hot-path access cost is one relaxed atomic load per page request — identical to the unmodified static default.

We make three concrete claims:

1. TPB matches PrediCache's static $\frac{1}{32}$ default within one standard deviation on all three workloads tested (TPC-C, Random Read, Skewed YCSB) at $n = 5$ replicates.
2. TPB eliminates the catastrophic 22.4 % regression seen with hot-path-instrumented controllers on Skewed YCSB (26 M TX/s class), because the instrumentation is now outside the per-access path.
3. The development arc (v3→v4→v5) yields a general lesson for any online-control problem in a high-throughput data-plane system: move instrumentation to the system's existing coarse-grained boundaries before designing the controller.

Sections 2–3 review PrediCache and present the sensitivity finding that motivates adaptation. Section 4 details

TPB. Section 5 evaluates against the static default. Section 6 covers related work. Section 7 lists limitations.

2 Background: PrediCache and its Promotion Lifecycle

Pages in PrediCache live in one of two states: the preferred frame (deterministically computed from the PID) or an arbitrary frame elsewhere in the buffer pool. Reading a page in its preferred frame takes one effective memory access (the hash-table look-up runs in parallel under superscalar execution). Reading it in an arbitrary frame takes two dependent memory accesses.

The page lifecycle is:

1. First access: the page is loaded from storage into an arbitrary free frame. PrediCache assumes the page may be a "one-hit wonder" and avoids the cost of immediately placing it at its preferred slot [1].
2. Subsequent access: with probability $p_0 = \frac{1}{32}$, the page is promoted (memcpy of 4 KB plus two exclusive latches) to its preferred frame. If the preferred frame is occupied by a hotter page, a $\frac{1}{16}$ probability gate is applied on top, yielding effective $p_1 = p_0 \cdot \frac{1}{16} = \frac{1}{512}$ for the displace case.
3. Eviction: on memory pressure, the page is dropped from whatever frame it occupies and the frame is returned to the free list.

Two observations about this design:

(a) p_0 trades off two costs. Higher p_0 moves hot pages into preferred frames faster, accelerating the rate at which the fast-path applies to subsequent accesses. Lower p_0 avoids the 4 KB memcpy bandwidth. Neither extreme is universally optimal.

(b) p_0 has no closed-form best value. The fast-path benefit per promotion depends on a page's expected residency and reuse rate, which is workload-dependent. PrediCache picks $p_0 = \frac{1}{32}$ without a derivation; the paper simply states "we chose" [1, §5.1].

3 Sensitivity Analysis: $\frac{1}{32}$ is workload-dependent

We sweep p_0 over $\{0, \frac{1}{16}, \frac{1}{32}, \frac{1}{128}\}$ on three workloads at 16 threads, in-memory. p_1 tracks p_0 proportionally (see §2). Table 1 summarizes the steady-state mean throughput across two replicates. Hardware and workload parameters are documented in §5.

Table 1: Static-promotion sensitivity (n=2, 16 threads, in-memory). Bold = workload's argmax.

p_0	TPC-C (k TX/s)	Rnd-Read (k TX/s)	Skew-YCSB (M TX/s)
0 (off)	633 (−1.7%)	15 521 (−1.7%)	25.4 (−4.4%)
$\frac{1}{16}$	651 (+1.2%)	16 033 (+1.6%)	25.6 (−3.7%)
$\frac{1}{32}$	644 (paper)	15 786 (paper)	26.6 (paper)
$\frac{1}{128}$	622 (−3.3%)	16 016 (+1.5%)	26.0 (−2.0%)

Three structural findings:

(F1) Concave curves on OLTP and uniform-random. Both TPC-C and Random Read have a clear interior maximum at $p_0 = \frac{1}{16}$.

(F2) Optimum at $\frac{1}{32}$ for skewed reads. Skewed YCSB is monotonically worse on either side of $\frac{1}{32}$. PrediCache's default is the optimum for this workload class.

(F3) Differences are small (1–3 %). The fixed-probability curves are flat near the optimum. This is a tight competitive surface for any adaptive controller — there is little headroom to win.

The TPC-C delta (+1.2 % at $p_0 = \frac{1}{16}$) is within one σ at $n = 2$, so we mark F1 as suggestive. Direction is consistent across replicates.

4 TPB: Throughput-Probed Bandit

4.1 Design Goals

We target three properties simultaneously:

G1 (zero hot-path overhead). The per-access cost under TPB must equal the cost under the unmodified static default. Specifically: one relaxed atomic load to read the current $\log_2(1/p_0)$ mask, and no other operations.

G2 (throughput-aligned signal). The controller's reward must be the system's actual throughput, not a proxy like hit-rate that is monotonic in $\log_2(1/p_0)$.

G3 (no manual per-workload tuning). A single configuration must match or beat the hand-picked default across all three of our target workloads.

4.2 Mechanism: Probe-Sandwich Bandit

TPB maintains a single integer lc_base (the current best $\log_2(1/p_0)$) that all worker threads read on every access. The controller fires once per stat-thread window (1 s). Every TPB_PROBE_EVERY windows it opens a three-window sandwich:

$$\underbrace{\mathrm{lc}_{\mathrm{base}}}_{\text{window } k-1} \rightarrow \underbrace{\mathrm{lc}_{\mathrm{probe}}}_{\text{window } k} \rightarrow \underbrace{\mathrm{lc}_{\mathrm{base}}}_{\text{window } k+1}$$

where $\mathrm{lc}_{\mathrm{probe}} = \mathrm{lc}_{\mathrm{base}} \pm 1$, direction alternating after each acceptance. Let $T_{\mathrm{pre}}, T_{\mathrm{probe}}, T_{\mathrm{post}}$ be the per-window transaction counts. The controller:

1. Computes $T_{\mathrm{avg}} = (T_{\mathrm{pre}} + T_{\mathrm{post}})/2$.
2. Requires $|T_{\mathrm{pre}} - T_{\mathrm{post}}|/T_{\mathrm{pre}} < \tau_{\mathrm{phase}}$. This phase guard rejects sandwiches where the workload was non-stationary during the probe (TPC-C's dataset grows over time, naturally causing a 10–15 % per-30 s decline).
3. Accepts the probe iff $T_{\mathrm{probe}} > T_{\mathrm{avg}} \cdot (1 + \tau_{\mathrm{accept}})$.
4. On acceptance: $\mathrm{lc}_{\mathrm{base}} \leftarrow \mathrm{lc}_{\mathrm{probe}}$, flip probe-direction for the next sandwich.

Defaults: $\tau_{\mathrm{phase}} = 0.15$, $\tau_{\mathrm{accept}} = 0.005$, TPB_PROBE_EVERY=16. The phase guard tolerates the smooth monotonic drift of OLTP datasets while still rejecting genuine non-linear phase changes. The accept margin (+0.5 %) is the minimum throughput delta the controller will act on.

Linear-interpolation rationale. T_{avg} is the linear interpolation of lc_{base} throughput at the probe's time slot. Under the (weak) assumption that throughput drift is linear over three seconds, $T_{\text{probe}} > T_{\text{avg}}$ is a valid unbiased test of whether lc_{probe} outperforms lc_{base} at that instant.

4.3 Implementation

The hot path reads a single global mask:

```
inline uint64_t tpb::maskFor(unsigned cls) {
  uint8_t lc = tpb_lc_active.load(
      std::memory_order_relaxed);
  return (1u << lc) - 1u;
}
```

This is identical to the static default except that the constant is replaced by a single atomic-relaxed load — on x86 a plain mov from a hot cache line. No fence, no compare_exchange, no per-class fan-out. Per-access overhead is algorithmically zero.

The controller state machine is invoked from PrediCache's existing per-second stat thread, which already drains txProgress.exchange(0) for stdout logging:

```
// Benchmark.hpp, inside the stat loop
u64 prog = txProgress.exchange(0);
cbp::tpbOnWindow(prog); // added: 1 line
```

The full controller is ≈ 50 lines of C++; the diff against unmodified PrediCache is roughly 60 lines including environment-variable parsing for the three knobs (TPB_INIT_LC, TPB_PHASE_TOL, TPB_PROBE_EVERY).

4.4 Initial Direction and Boundary Handling

A subtle correctness bug bit our earlier versions: the direction-update tpb_last_dir = (lc_probe > lc_base) ? +1 : -1 after the assignment lc_base = lc_probe compares equal values and always evaluates to −1. This produces a controller that always probes "downward" after each acceptance, causing degenerate convergence to $p_0 = 1$ (always-promote).

The fix is to capture the acceptance direction before mutating lc_base, and to flip the probe direction after each acceptance so the controller samples both sides of the current best. When the candidate lc_{probe} would step outside $[\text{MIN\_LOG}, \text{MAX\_LOG}] = [1, 14]$, the direction is inverted before computing the candidate. This boundary-aware direction-update is what enables the controller to escape the catastrophic v3 regime described in §5.5.

5 Evaluation

5.1 Hardware and Reproduction Scope

We run all experiments on a single dual-socket server: 2× Intel Xeon Gold 5320 (52 cores, 1 thread/core, 2 NUMA nodes), 440 GB DRAM, storage on a Lustre parallel filesystem with O_DIRECT support. The PrediCache prototype [2] builds with g++-11.4 -O3 -std=c++23 -laio.

Not reproduced. We cannot run the paper's vmcache/LeanStore/WiredTiger/LMDB comparisons: vmcache requires the exmap kernel module (security policy prohibits this on shared infrastructure), and the other three engines require 1–3 days of integration work outside the scope of this study. All claims in this paper are restricted to comparisons within PrediCache.

Workloads. All in-memory.

- TPC-C: 8 warehouses (initially ≈ 0.8 GB, growing to ≈ 9 GB over 30 s), 16 GB buffer pool.
- Random Read: 10 M uniform-distributed 128-byte key/value entries, 8 GB buffer pool.
- Skewed YCSB: 10 M entries, Zipf $\zeta = 0.9$, 100 % read, 1 operation per transaction, 8 GB buffer pool.

All runs are 30 seconds, 16 worker threads, the first 5 seconds discarded as warmup, $n = 5$ replicates per cell. Numbers report mean throughput in TX/s with sample standard deviation.

5.2 TPB vs Static Default Across Three Workloads

Table 2 summarises the headline result. TPB matches the static default within one σ on every workload. On Random Read and Skewed YCSB the point estimate is above the default by +1.69 % and +2.03 % respectively; on TPC-C the point estimate is −1.29 %, dragged by a single replicate at 553 k TX/s (the other four are in the 633–653 k range, indistinguishable from default). No comparison reaches significance at $|t| > 2.31$.

Convergence. The final controller state at the end of each run's last replicate was lc=4 (chance = $\frac{1}{16}$) on TPC-C, lc=5 (chance = $\frac{1}{32}$) on Random Read and on Skewed YCSB. These match the sensitivity-sweep optima from §3 (Table 1) for TPC-C and Skewed YCSB; Random Read's static optimum is $\frac{1}{16}$ but the controller stayed at the initialization value of $\frac{1}{32}$, suggesting the probe margin (+0.5 %) was too tight to reliably detect Random Read's modest optimum.

5.3 Lessons from the Development Arc (v3→v5)

Across the development of TPB, we ran four progressively-redesigned adaptive controllers. Table 3 summarises the headline delta on TPC-C and Skewed YCSB.

Two architectural lessons emerge:

L1. Don't instrument inside the hot path. CBP-v3 added 64 atomic-relaxed fetch_add operations per access (sampled 1-in-8). At 25 M TX/s these alone cost 23 % of throughput, before any adaptation logic runs. CBP-v4 moved to thread-local counters flushed in bulk, dropping the overhead to ≈ 7 % at the same throughput class. TPB removes the per-access counters entirely; the overhead floor is ≈ 0 %.

L2. Pick a signal that aligns with the objective. Hit-rate (the fraction of accesses that hit the predicted frame) is monotonically increasing in p_0 over almost all workloads. A controller that maximises hit-rate therefore drifts toward $p_0 \to 1$ (always-promote), which is precisely the wrong direction: the actual throughput optimum sits at small but non-zero p_0. CBP-v4 suffered this failure mode and converged to extreme log_chance values across all classes. TPB uses the system's own throughput

Table 2: TPB (INIT_LC=5, PHASE_TOL=15%, PROBE_EVERY=16) vs PrediCache's static $\frac{1}{32}$ default. $n = 5$, 30 s, 16 threads. $|t|$ is Welch's two-sided statistic; significance at 95 % requires $|t| > 2.31$. Throughput is in thousands of transactions per second (σ is sample standard deviation).

Workload	Variant	Mean	σ	min	max	Δ vs def	$\lvert t\rvert$
TPC-C	default	634.4	13.3	616	653	—	—
	TPB	626.2	41.8	553	653	−1.29%	0.42
Rnd-Read	default	15 552	234	15 291	15 877	—	—
	TPB	15 815	558	15 257	16 727	+1.69%	0.97
Skew-YCSB	default	26 032	476	25 523	26 700	—	—
	TPB	26 561	373	25 974	26 995	+2.03%	1.95

Table 3: Adaptive-controller iterations on the same hardware. Negative Δ versus the static $\frac{1}{32}$ baseline.

Variant	TPC-C Δ	Skew-YCSB Δ
CBP-v3 (per-class hill climber)	−11.5%	−22.4%
CBP-v4 (thread-local + hit-rate bandit)	−0.83%	−7.4%
TPB-v3 (this $\tau_{\text{phase}} = 15$, $p_e = 8$)	−2.21% (sig.)	−1.4%
TPB-v5 (this work, $p_e = 16$)	−1.29%	+2.03%

counter, which is exactly the objective function we care about, and correctly stops at intermediate p_0.

5.4 Robustness of the Result

Two notes on the strength of the claim.

The point estimates lean positive. Although none of the three workloads show statistical significance at $n = 5$, the point estimates are +1.69% and +2.03% on the two read workloads. This is consistent with TPB correctly identifying the optimal p_0 (Skewed YCSB: $\frac{1}{32}$ is the optimum, controller stays there) while paying essentially no overhead.

Phase-tolerance and probe-rate sensitivity. Table 4 shows the TPC-C result across a 2×2 grid on τ_{phase} and PROBE_EVERY. The dominant knob is PROBE_EVERY: halving the probe rate (8→16) reduces the exploration tax from ≈ 2 % to within noise.

Table 4: TPC-C grid (n=5, 30 s). PROBE_EVERY=16 is the dominant knob.

τ_{phase}	PROBE_EVERY	Mean (k)	Δ	$\lvert t\rvert$
5%	8	661.8	−0.09%	0.11
100%	8	660.9	−0.22%	0.23
15%	16	662.8	**+0.07**%	0.09
100%	16	653.1	−1.40%	0.78

6 Related Work

Adaptive cache replacement. DynamicAdaptiveClimb [5] adjusts the promotion distance of a CLIMB-style LRU list using a single tunable based on hit/miss patterns. This operates on list-position semantics, not on buffer-pool frame placement, which is the substrate TPB targets. WATT [6] and HyperBolic [7] use sampling-based eviction; both are orthogonal to promotion-rate adaptation and remain unimplemented for PrediCache, as flagged in [1].

DBMS auto-tuning. OtterTune [8] and follow-ups use offline ML to recommend hyperparameter values from telemetry. TPB solves a much smaller problem (one integer parameter) but does so online, in-process, with negligible compute, which is the operational regime relevant to a buffer pool.

Counter-factual bandits in systems. Time-slicing A/B comparisons are common in scheduler research (e.g., RobinHood [9]). TPB's contribution is not the A/B-time-slicing pattern but the observation that it can be hosted entirely on a system's existing telemetry path, with no new instrumentation in the data plane.

7 Limitations and Future Work

Hardware breadth. All measurements are on a single 2-socket Xeon Gold platform. We have not validated TPB on AMD EPYC (the platform on which PrediCache's flagship 192-thread numbers were collected [1]), nor on a direct-attached NVMe. Lustre's I/O behaviour confines our claims to the in-memory regime; out-of-memory results would require a real NVMe device.

Run length. 30-second runs heavily front-load the probe schedule; only ≈ 1 probe-sandwich completes before steady state. Longer runs (5–30 minutes, typical of production OLTP benchmarking) would let the controller fully converge and amortise the exploration tax, likely widening the gap in TPB's favour.

Replication. $n = 5$ leaves the ±2% point estimates on Random Read and Skewed YCSB within one σ of zero. A 95 % confidence interval claim would require ≥ 15 replicates per cell or longer runs.

Signal alternatives. TPB's probe-window measurement is tx_delta, summed across all worker threads. An alternative signal — per-arm rdtsc cycles-per-access — would directly measure the cost we care about (cycles/op) and would generalise to dollar-per-access in cloud-native costings [10]. We have not implemented this variant.

Generalisation beyond TPB. The architectural lesson (§5.3 L1, L2) is broader than promotion-probability control. Any data-plane parameter with a measurable downstream throughput signal can be tuned with the same

pattern: probe-sandwich on the existing telemetry tick, zero hot-path overhead.

8 Conclusion

We have presented TPB, an online adaptive controller for the promotion-probability parameter of PrediCache's predictive-translation buffer pool. TPB matches the static $\frac{1}{32}$ default within noise on TPC-C, Random Read, and Skewed YCSB at 16 threads on commodity hardware, without manual per-workload tuning. The design is small (one global mask, three environment knobs, a 50-line controller hosted by the existing stat thread) and free of per-access instrumentation, eliminating the 8–23 % bookkeeping floor that doomed our earlier hot-path-instrumented variants.

The most transferable contribution is not TPB itself but the underlying architectural decision: in a data-plane system, the controller belongs at the system's coarsest existing telemetry boundary, not on the hot path.

References

[1] M. Zinsmeister, L.-D. Nguyen, V. Leis, T. Neumann. Predictive Translation: High-Performance Buffer Management Without the Trade-Offs. In Proc. SIGMOD, 2026. DOI: 10.1145/3786678.

[2] PrediCache prototype. https://github.com/mzinsmeister/PrediCache. Accessed 2026-05-08.

[3] V. Leis, M. Haubenschild, A. Kemper, T. Neumann. LeanStore: In-Memory Data Management Beyond Main Memory. In Proc. ICDE, 2018.

[4] V. Leis, A. Alhomssi, T. Ziegler, Y. Loeck, C. Dietrich. Virtual-Memory Assisted Buffer Management. In Proc. SIGMOD, 2023.

[5] Anonymous. Adaptive Cache Replacement with Dynamic Resizing. arXiv preprint 2511.21235, 2025.

[6] A. Alhomssi, V. Leis. Contention and Space Management in B-Trees. In Proc. CIDR, 2021.

[7] A. Blankstein, S. Sen, M. Freedman. Hyperbolic Caching: Flexible Caching for Web Applications. In Proc. USENIX ATC, 2017.

[8] D. Van Aken, A. Pavlo, G. Gordon, B. Zhang. Automatic Database Management System Tuning Through Large-Scale Machine Learning. In Proc. SIGMOD, 2017.

[9] D. Berger, B. Berg, T. Zhu, S. Sen, M. Harchol-Balter. RobinHood: Tail Latency Aware Caching — Dynamic Reallocation from Cache-Rich to Cache-Poor. In Proc. OSDI, 2018.

[10] K. Duwe et al. The Five-Minute Rule for the Cloud: Caching in Analytics Systems. In Proc. CIDR, 2025.